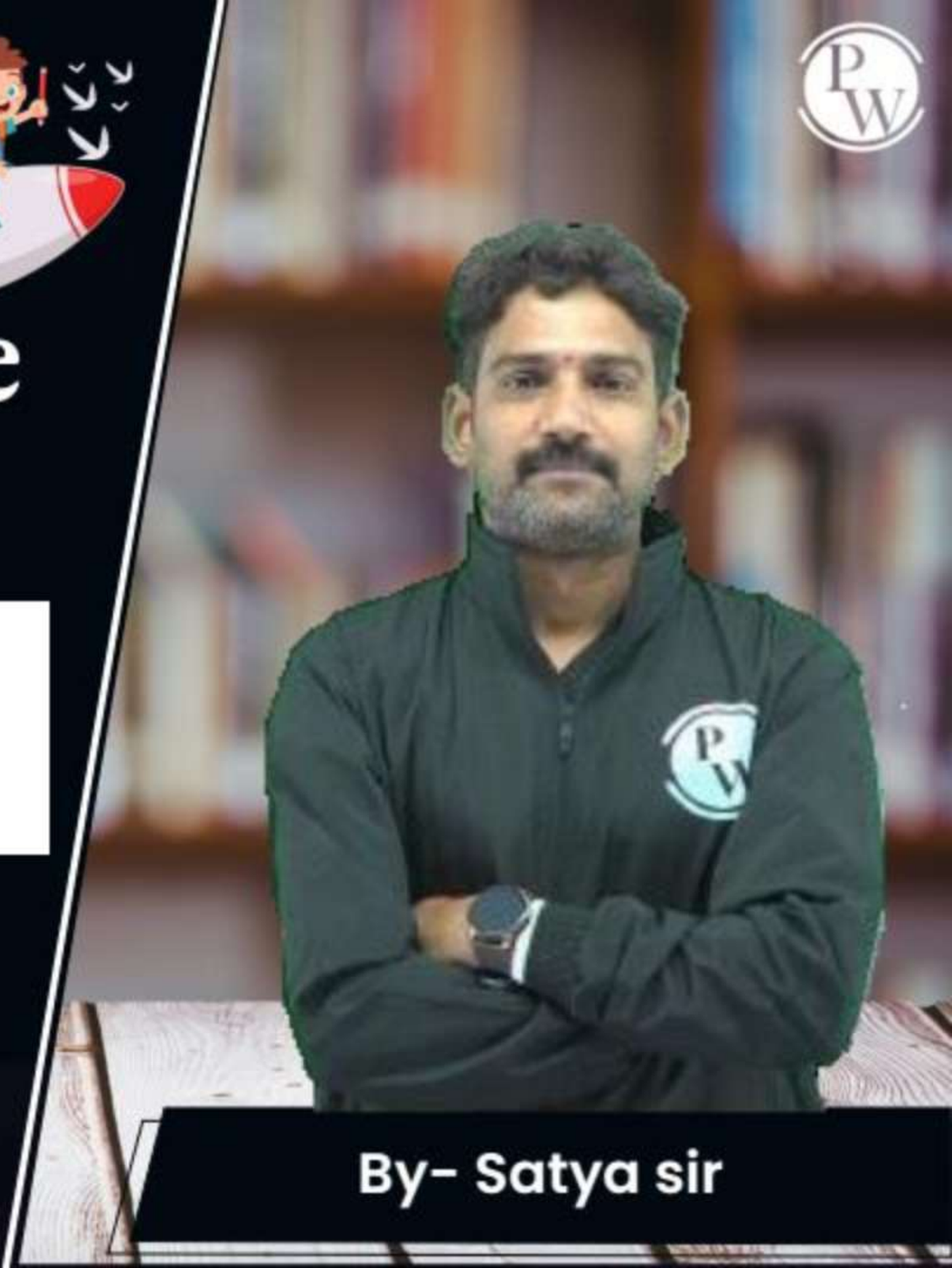
Data Science & Artificial Intelligence

Data Structures Through Python

HASH TABLES

Lecture No.- 02



By- Satya sir

Recap of Previous Lecture

Hashing

- Hash Function
- Collission ⇒ more than one key maps to the same value
- Collission Resolution Techniques:
  - Open Hashing | Chaining method ✓
  - Closed Hashing | Open addressing
    - Linear Probing
    - Quadratic Probing
    - double Hashing

Topics to be Covered

Open addressing Techniques

- Linear Probing
- Quadratic Probing
- Double Hashing

Topic : Hash Tables - 2

Primary clustering

Consecutive slots form as cluster and Increase number of Probes for a key to find free slot.

Linear Probing :

Probe = Attempt

Hash Function : $\boxed{h(k, i) = (k + i) \% N}$

$i = 0, 1, 2, 3 \ldots$ for Each 'K'

N = Table size / Number of slots

i = Probe Number

Ex: Keys = {25, 33, 44, 14, 20, 49} Table size = 6

$h(25,0) = (25+0) \% 6 = 1$

$h(33,0) = (33+0) \% 6 = 3$

$h(44,0) = (44+0) \% 6 = 2$

$h(14,0) = (14+0) \% 6 = 2$ collission

$h(14,1) = (14+1) \% 6 = 3$ collission

$h(14,2) = (14+2) \% 6 = 4$

$h(20,0) = 20 \% 6 = 2$ Collission

$h(20,1) = (20+1) \% 6 = 3$ collission

$h(20,2) = (20+2) \% 6 = 4$ collision

$h(20,3) = (20+3) \% 6 = 5$

$h(49,0) = 49 \% 6 = 1$

$h(49,1) = (49+1) \% 6 = 2$

$h(49,2) = (49+2) \% 6 = 3$

$h(49,3) = (49+3) \% 6 = 4$

$h(49,4) = (49+4) \% 6 = 5$

$h(49,5) = (49+5) \% 6 = 0$

0	49
1	25
2	44
3	33
4	14
5	20

cluster

Topic : Hash Tables - 2

$h(49,3) = (1+3^2) \% 6$
$= 10 \% 6 = 4$

Quadratic Probing

$$h(K,i) = (h'(K) + i^2) \% N \qquad i = 0,1,2,3,\ldots \text{ for Each key}$$

$$h'(K) = K \% N$$

keys $= \{25, 33, 44, 14, 20, 49\}$ $\quad N = 6$

$h(25,0) = ((25 \% 6) + 0) \% 6$
$= (1+0) \% 6 = 1$

$h(33,0) = ((33 \% 6) + 0) \% 6$
$= 3$

$h(44,0) = ((44 \% 6) + 0) \% 6$
$= 2$

$h(14,0) =$ 2 collission

$h(14,1) = ((14 \% 6) + 1^2) \% 6$
$= 3$ collission

$h(14,2) = ((14 \% 6) + 2^2) \% 6$
$= (2+4) \% 6 = 0$

$h(20,0) =$ 2 collision

$h(20,1) =$ 3 collision

$h(20,2) = ((20 \% 6) + 2^2) \% 6$
$= 6 \% 6 = 0$

$h(20,3) = (2+3^2) \% 6$
$= (2+9) \% 6$
$= 11 \% 6$
$= 5$

$h(49,0) =$ 1 collision

$h(49,1) =$ 2 collision

$h(49,2) = (1+4) \% 6 = 5$ collision

0	14
1	25
2	44
3	33
4	49
5	20

Topic : Hash Tables - 2

1, 2, 5, 4

- In Quadratic Probing, The Number of Probes needed for a key will be Less as Compared to Linear Probing.

- In Quadratic Probing, Secondary clustering issue occur.

$h(61,0) = 1$
$h(61,1) = 2$
$h(61,2) = 5$
$h(61,3) = 4$
$h(61,4) = 5$
$h(61,5) = 2$
$h(61,6) = 1$

$h(73,0) = 1$
$h(73,1) = 2$
$h(73,2) = 5$
$h(73,3) = 4$
$h(73,4) = 5$
$h(73,5) = 2$
$h(73,6) = 1$

$h(19,0) = 1$
$h(19,1) = 2$
$h(19,2) = 5$
$h(19,3) = 4$
$h(19,4) = 5$
$h(19,5) = 2$
$h(19,6) = 1$

Slots 1, 2, 4, 5 forming as cluster

Keys will be looped over slots in the cluster.

(Non-Consecutive slots forming as a cluster)

Topic : Hash Tables - 2

Double Hashing (Free from clustering issue)

$$h(k, i) = \left(h_1(k) + i\, h_2(k)\right) \% N$$

$h_1(k) = k \% N$

$h_2(k) = k \% N_1$

$i = 0, 1, 2, \ldots$ for Each key

N_1 is Smaller Prime than N

$i = 0$ $\quad h(k) = \left(h_1(k) + 0 * h_2(k)\right) \% N$

$= (h_1(k)) \% N$

$= (k \% N) \% N$

$= k \% N == h_1(k)$

Keys = {25, 33, 44, 14, 20, 49, 61, 73, 19} $N = 6$

$N_1 = 3$

$h(25,0) = 25 \% 6 = 1$

$h(33,0) = 33 \% 6 = 3$

$h(44,0) = 44 \% 6 = 2$

$h(14,0) = 14 \% 6 = \underline{2}$

$h(14,1) = \left(h_1(14) + 1 \cdot h_2(14)\right) \% 6$

$= \left[2 + 1 * (14 \% 3)\right] \% 6$

$= (2+2) \% 6$

$= \underline{4}$

$h(20,0) = 20 \% 6 = \underline{2}$

$h(20,1) = (2 + 1 * 20 \% 3) \% 6$

$= (2+2) \% 6 = \underline{4}$

$h(20,2) = \left[2 + 2 * 20 \% 3\right] \% 6$

$= (2+4) \% 6$

$= \underline{0}$

$h(49,0) = 1$

$h(49,1) = (1+1) \% 6 = \underline{2}$

$h(49,2) = (1 + 2 * 1) \% 6$

$= \underline{3}$

$h(61,0) = 1$

$h(61,1) = (1+1) \% 6 = 2$

$h(61,2) = 3$

$h(61,3) = (1+9) \% 6$

$= 4$

Topic : Hash Tables - 2

#Q. Consider a double hashing scheme in which the primary hash function is h1(k) = k mod 21, and the secondary hash function is h2(k) = 1+(k mod 19). Assume that the table size is 21. Then the address returned by probe 1 in the probe sequence (assume that the probe sequence begins at probe 0) for key value k = 70 is _____ .

$$h(k,i) = [h1(k) + i * h2(k)] \% N$$

A. 0 B. 1 C. 2 D. 4

$$h(70,1) = [h_1(70) + 1 * h2(70)] \% 21$$

$$= [(70 \% 21) + 1 * (1 + (70 \% 19))] \% 21$$

$$= [7 + 1 * (1 + 13)] \% 21$$

$$= (7 + 14) \% 21 = 21 \% 21 = 0$$

Ex:

Keys = {79, 69, 98, 72, 14, 40, 27}

$N = 13$

$h_1(K) = K \% 13$

$h_2(K) = [1 + (K \% 11)]$

$h(79,0) = 79 \% 13 = 1$

$h(69,0) = 69 \% 13 = 4$

$h(98,0) = 98 \% 13 = 7$

$h(72,0) = 72 \% 13 = 7$ Collision

$h(72,1) = [(72 \% 13) + 1 * [1 + (72 \% 11)]] \% 13$

$= [7 + 1 * [1 + 6]] \% 13$

$= (7+7) \% 13 = 1$ Collision

$h(72,2) = (7 + 2 * 7) \% 13$

$= 21 \% 13 = 8$

$h(14,0) = 1$ Collision

$h(14,1) = [1 + 1 * [1 + (14 \% 11)]] \% 13$

$= (1+4) \% 13 = 5$

$h(40,0) = 1$ Collision

$h(40,1) = [1 + 1 * [1 + (40 \% 11)]] \% 13$

$= (1+8) \% 13$

$= 9$

$h(27,0) = 1$ Collision

$h(27,1) = [1 + 1 * [1 + (27 \% 11)]] \% 13$

$= (1+6) \% 13 = 7$ Collision, $h(27,2) = (1+12) \% 13 = 0$

0	27
1	79
2	
3	
4	69
5	14
6	
7	98
8	72
9	40
10	
11	
12	

Topic : Hash Tables - 2

#Q. A hash table contains 10 buckets and uses linear probing to resolve collisions. The key values are integers and the hash function used is key % 10. If the values 43, 165, 62, 123, 142 are inserted in the table, in what location would the key value 142 be inserted?

(A) 2
(B) 3
(C) 4
(D) 6 ✓

$h(43,0) = 43 \% 10 = 3$

$h(165,0) = 5$

$h(62,0) = 2$

$h(123,0) = 3$ Collission

$h(123,1) = (123+1) \% 10$
$= 4$

$h(142,0) = 2$
$h(142,1) = 3$
$h(142,2) = 4$
$h(142,3) = 5$
$h(142,4) = 6$

0	
1	
2	62
3	43
4	123
5	165
6	142
7	
8	
9	

Topic : Hash Tables - 2

2 4 1 9 9 1 3 9

#Q. Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function x mod 10, which of the following statements are true? [Chaining method]

1. 9679, 1989, 4199 hash to the same value ✓
2. 1471, 6171 hash to the same value ✓
3. All elements hash to the same value ✗
4. Each element hashes to a different value ✗

(A) 1 only
(B) 2 only
(C) 1 and 2 only ✓
(D) 3 or 4

Topic : Hash Tables - 2

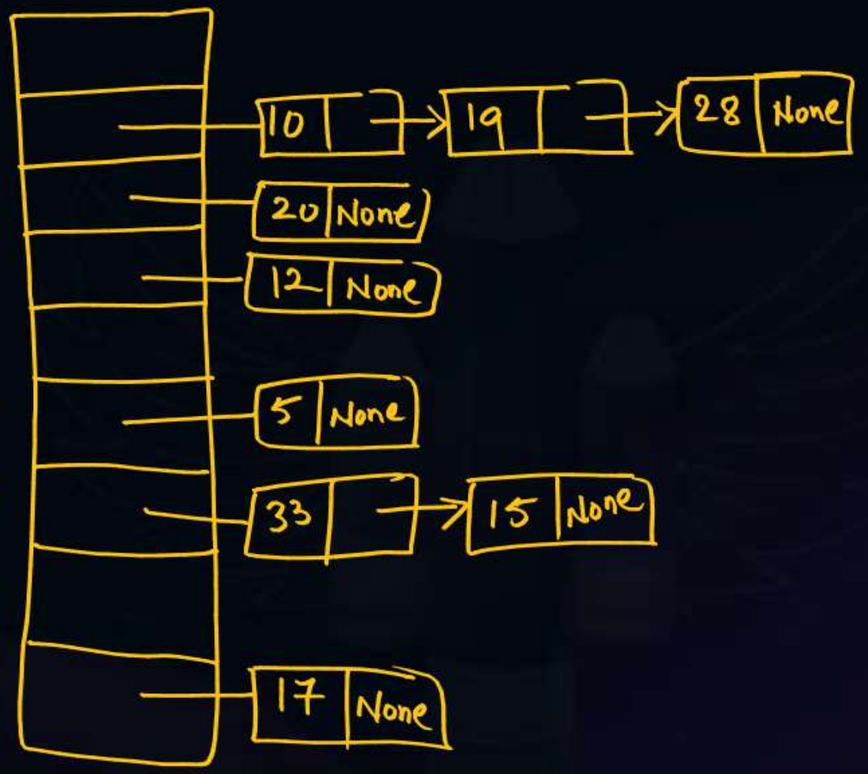
#Q. Consider a hash table with 9 slots. The hash function is $h(k) = k \bmod 9$. The collisions are resolved by chaining. The following 9 keys are inserted in the order: 5, 28, 19, 15, 20, 33, 12, 17, 10. The maximum, minimum, and average chain lengths in the hash table, respectively, are

(A) 3, 0, and 1
(B) 3, 3, and 3
(C) 4, 0, and 1
(D) 3, 0, and 2

$h(5) = 5$
$h(28) = 1$
$h(19) = 1$
$h(15) = 6$
$h(20) = 2$
$h(33) = 6$
$h(12) = 3$
$h(17) = 8$
$h(10) = 1$

Max = 3
min = 0
Avg = $\frac{9}{9} = 1$

Chain length	
0	0
3	1
1	2
1	3
0	4
1	5
2	6
0	7
1	8



Topic : Hash Tables - 2

#Q. A hash table of length 10 uses open addressing with hash function h(k)=k mod 10, and linear probing. After inserting 6 values into an empty hash table, the table is as shown below.

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Which one of the following choices gives a possible order in which the key values could have been inserted in the table?

(A) 46, 42, 34, 52, 23, 33
(B) 34, 42, 23, 52, 33, 46
(C) 46, 34, 42, 23, 52, 33
(D) 42, 46, 33, 23, 34, 52

A)

0	
1	
2	42
3	52
4	34
5	23
6	46
7	33
8	
9	

B)

0	
1	
2	42
3	34
4	23
5	52
6	33
7	46
8	
9	

C)

0	
1	
2	42
3	23
4	34
5	52
6	46
7	33
8	
9	

Topic : Hash Tables - 2

Please go through, for idea

```
ht = HashTable(5)

 ht.insert("apple", 3)
  ht.insert("banana", 2)
  ht.insert("cherry", 5)

print("apple" in ht)  # True
 print("durian" in ht)  # False

print(ht.search("banana"))  # 2

  ht.insert("banana", 4)
  print(ht.search("banana"))  # 4

  ht.remove("apple")
  print(len(ht))  # 3
```

Topic : Hash Tables - 2

```
class Node:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None


class HashTable:
    def __init__(self, capacity):
        self.capacity = capacity
        self.size = 0
        self.table = [None] * capacity

    def _hash(self, key):
        return hash(key) % self.capacity
```

```
def insert(self, key, value):
        index = self._hash(key)

        if self.table[index] is None:
            self.table[index] = Node(key, value)
            self.size += 1
        else:
            current = self.table[index]
            while current:
                if current.key == key:
                    current.value = value
                    return
                current = current.next
            new_node = Node(key, value)
            new_node.next = self.table[index]
            self.table[index] = new_node
            self.size += 1
```

Topic : Hash Tables - 2

```
def search(self, key):
    index = self._hash(key)

    current = self.table[index]
    while current:
        if current.key == key:
            return current.value
        current = current.next

    raise KeyError(key)
```

```
def __len__(self):
    return self.size

def __contains__(self, key):
    try:
        self.search(key)
        return True
    except KeyError:
        return False
```

```
def remove(self, key):
    index = self._hash(key)

    previous = None
    current = self.table[index]

    while current:
        if current.key == key:
            if previous:
                previous.next = current.next
            else:
                self.table[index] = current.next
            self.size -= 1
            return
        previous = current
        current = current.next

    raise KeyError(key)
```

2 mins Summary

- Open addressing Techniques
  - Linear Probing
  - Quadratic Probing
  - Double Hashing.

THANK - YOU